



Université de
Sherbrooke

Université de Sherbrooke

Faculté des sciences

Département d'informatique

Maîtrise en informatique cheminement Cybersécurité

Travail pratique – Implémentation du SSO avec Authelia

Groupe nom-groupe, les membres

Augey, Louis - augl3254

Etheve, Alexandre - etha7814

Kaba, Mohamed -

Laumonier, Robin - laur4848

Sonzogni, Matthieu - sonm3513

Travail présenté à : Mr Romain Levifve

Dans le cadre du cours : INF806-H26 – Système et réseau

Date : 20 avril 2026

Table des matières

I – INTRODUCTION.....	4
1.1 – Contexte et objectifs du projet.....	4
1.2 – Présentation de TechSolutions.....	4
1.3 – Périmètre du projet.....	5
II – ÉTAT DE L'ART ET TECHNOLOGIES EMPLOYÉES.....	6
2.1 – Concept SSO (Single Sign-On).....	6
2.2 – OpenID Connect 1.0 (OIDC).....	6
2.2.1 – Architecture et acteurs.....	7
2.2.2 – Authorization Code Flow.....	7
2.2.3 – Les tokens JWT (JSON Web Token).....	8
2.2.4 – Implémentation d'OIDC.....	8
2.3 – Authelia – Le cœur du système.....	8
2.4 – Structure des dossiers.....	10
2.5 – Nginx comme reverse proxy.....	10
2.6 – Sécurité des systèmes SSO.....	10
III – ORGANISATION DU GROUPE.....	12
3.1 – Répartition des rôles et attribution des tâches.....	12
3.2 – Outils de collaboration.....	12
IV – ARCHITECTURE DE L'INFRASTRUCTURE.....	13
4.1 – Segmentation réseau Docker.....	13
4.2 – FQDN et plan d'adressage.....	14
4.3 – Diagramme de flux réseau et ports.....	14
4.4 – Choix technique et justification.....	14
V – GESTION DES IDENTITÉS ET DES ACCÈS.....	16
5.1 – Nomenclature CIP des utilisateurs.....	16
5.2 – Annuaire OpenLDAP, Groupes et permissions associées.....	16
5.3 – Politique d'accès Authelia.....	17
VI – IMPLÉMENTATION.....	18
6.1 – Mise en place de l'environnement Docker.....	18
6.2 – Configuration de Nginx.....	18
6.3 – Déploiement d'Authelia.....	18
6.4 – Intégration OIDC par service.....	19
6.4.1 – GitLab.....	19
6.4.2 – Jenkins.....	19
6.4.3 – Nextcloud.....	19
6.4.4 – Rocket.Chat.....	20
6.4.5 – LibreDesk.....	20

6.4.6 – Authelia.....	21
6.5 – Gestion des variables d’environnement et secrets.....	21
6.6 – Configuration du 2FA.....	21
VII – POINTS DÉPASSANT LE CAHIER DE CHARGES.....	23
7.1 – Gestion sécurisée des secrets par injection.....	23
VIII – POINTS BLOQUANTS ET MITIGATIONS.....	24
8.1 – Problèmes rencontrés.....	24
8.2 – Solutions apportées.....	24
IX – CONCLUSION.....	26
9.1 – Notions apprises.....	26
9.2 – Perspectives d'amélioration.....	26
X – RÉFÉRENCES.....	27
XI – Annexes.....	28
Annexe A — Arborescence du projet.....	28
Annexe B — Script d’initialisation Nextcloud `setup.sh`.....	29
Annexe C — Client type dans la configuration Authelia.....	29

I – INTRODUCTION

1.1 – Contexte et objectifs du projet

Dans un environnement numérique moderne, les organisations utilisent une multitude de services applicatifs tels que des outils de gestion de code, des plateformes de collaboration ou encore des systèmes de déploiement continu. Cette diversité pose un défi majeur en matière de gestion des identités et des accès.

L'utilisation de multiples identifiants et mots de passe pour différents services entraîne plusieurs problématiques tels que :

- Complexité pour l'utilisateur
- Augmentation de la surface d'attaque (réutilisation de mots de passe)
- Difficulté d'administration pour les équipes techniques
- Etc.

Afin de répondre à ces enjeux de sécurité et d'organisation, les organisations adoptent des mécanismes d'authentification centralisée, dont le Single Sign On (SSO) qui constitue une solution clé. Le SSO permet à un utilisateur de s'authentifier une seule fois pour accéder à plusieurs services, améliorant ainsi à la fois la sécurité et l'expérience utilisateur.

Ce présent projet s'inscrit dans ce cadre, et a pour objectif principal de concevoir et d'implémenter une infrastructure sécurisée reposant sur un système SSO basé sur le protocole OpenID Connect (OIDC), avec la solution open source Authelia

1.2 – Présentation de TechSolutions

TechSolutions est une entreprise de développement logiciel de taille moyenne spécialisée dans la conception d'applications web et la fourniture de solutions cloud pour divers clients. L'entreprise s'appuie sur une infrastructure intégrant divers outils nécessaires au cycle de vie des applications. Parmi les services utilisés, on retrouve notamment :

- Plateformes de gestion de code source et d'intégration continue – GitLab, Jenkins
- Une solution de stockage et de partage de fichiers – Nextcloud
- Une plateforme de communication et gestion tickets – Rocket.Chat, Libredesk

Ces services sont utilisés par différents profils d'utilisateurs (développeurs, administrateurs, chefs de projet, etc.), chacun ayant des besoins spécifiques en termes d'accès et de permissions.

C'est dans ce contexte que TechSolutions souhaite renforcer la sécurité de son infrastructure tout en simplifiant l'accès aux différents services en mettant en place d'un système SSO afin de centraliser l'authentification et faciliter la gestion des identités. Les objectifs sont donc :

- Mettre en place une architecture d'authentification centralisée
- Implémenter un mécanisme de SSO sécurisé avec authentification à deux facteurs

- Intégrer les services applicatifs *GitLab*, *Jenkins*, *LibreDesk*, *Nextcloud*, *Rocket.Chat* dans un écosystème unifié
- Garantir la sécurité des communications via l'utilisation de certificats TLS
- Modéliser et documenter les flux d'authentification

1.3 – Périmètre du projet

Le périmètre de ce projet couvre la conception, la mise en œuvre et la documentation d'une infrastructure SSO sécurisée intégrant cinq services applicatifs (GitLab, Jenkins, Nextcloud, LibreDesk, Rocket.Chat) :

- L'implémentation d'un système SSO basé sur Authelia
- L'utilisation du protocole OIDC pour l'authentification
- La sécurisation des accès via un reverse proxy – Nginx
- L'intégration des services applicatifs
- La gestion des utilisateurs, des groupes et des permissions
- L'activation de l'authentification à deux facteurs (2FA) pour tous les utilisateurs
- La sécurisation des communications via HTTPS

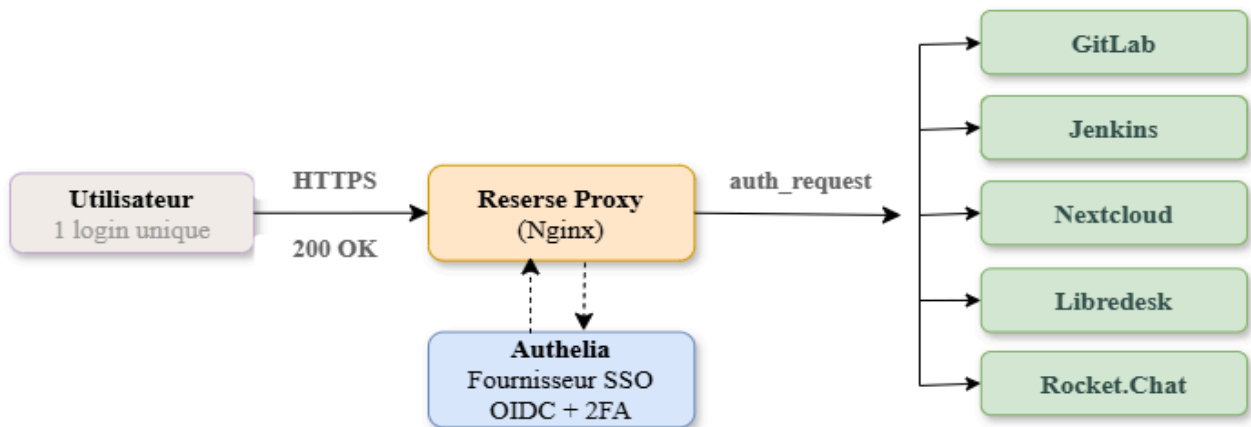


Figure 1 – Vue simplifiée de l'architecture SSO

II – ÉTAT DE L'ART ET TECHNOLOGIES EMPLOYÉES

2.1 – Concept SSO (Single Sign-On)

Le SSO est un mécanisme d'authentification permettant à un utilisateur d'accéder à plusieurs applications ou services après s'être authentifié une seule fois. Le SSO centralise l'authentification via un fournisseur d'identité unique (Authelia) qui authentifie l'utilisateur une seule fois, puis émet des jetons acceptés par tous les services connectés; Cela simplifie l'accès aux ressources tout en renforçant la sécurité globale du système.

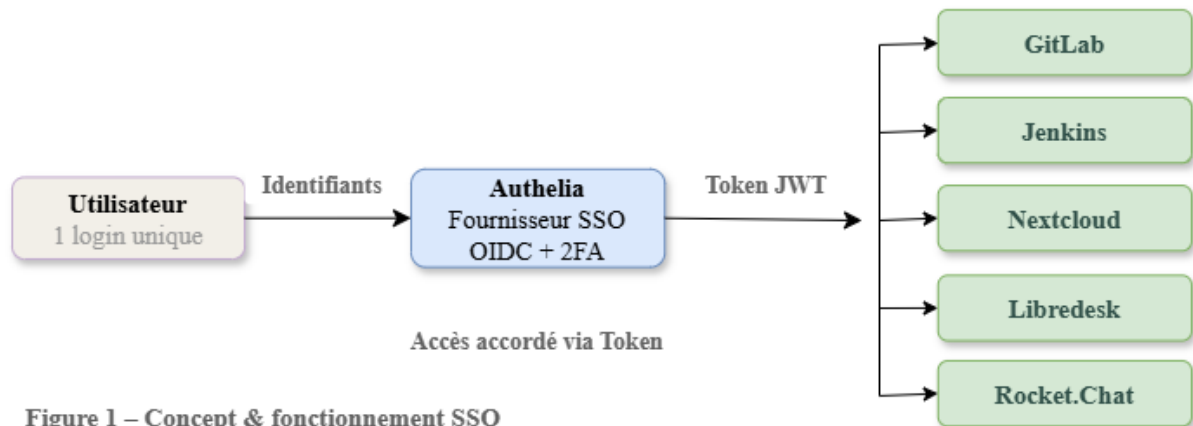


Figure 1 – Concept & fonctionnement SSO

Le SSO repose sur une relation de confiance entre :

- Le fournisseur d'identité – Authelia
- Les services consommateurs – GitLab, Jenkins, Nextcloud, LibreDesk, Rocket.Chat

Lorsqu'un utilisateur tente d'accéder à un service :

- 1. Le service vérifie si l'utilisateur est authentifié
- 2. Si ce n'est pas le cas, il redirige vers le fournisseur d'identité
- 3. L'utilisateur s'authentifie (mot de passe + 2FA)
- 4. Le fournisseur d'identité renvoie une preuve d'authentification (Token JWT)
- 5. L'utilisateur accède aux services sans nouvelle authentification

2.2 – OpenID Connect 1.0 (OIDC)

OIDC est le protocole standard qu'Authelia implémente. C'est une couche d'identité construite au-dessus d'OAuth 2.0. Il définit comment le fournisseur d'identité (Authelia) prouve à un service consommateur (GitLab, etc.) que l'utilisateur est bien qui il prétend être, en émettant des « **Tokens JWT signés** ».

Dans une architecture SSO, OIDC joue donc le rôle de protocole de communication entre le fournisseur d'identité (Authelia) et les services consommateurs (GitLab, Jenkins, etc); Il permet :

- De vérifier l'identité de l'utilisateur
- De transmettre des informations (nom, courriel, rôles, ...)
- De gérer les sessions de manière sécurisée

2.2.1 – Architecture et acteurs

Dans notre architecture – Authelia joue le rôle de « Authorization server », l'utilisateur qui souhaite accéder aux services est le « Ressource Owner »

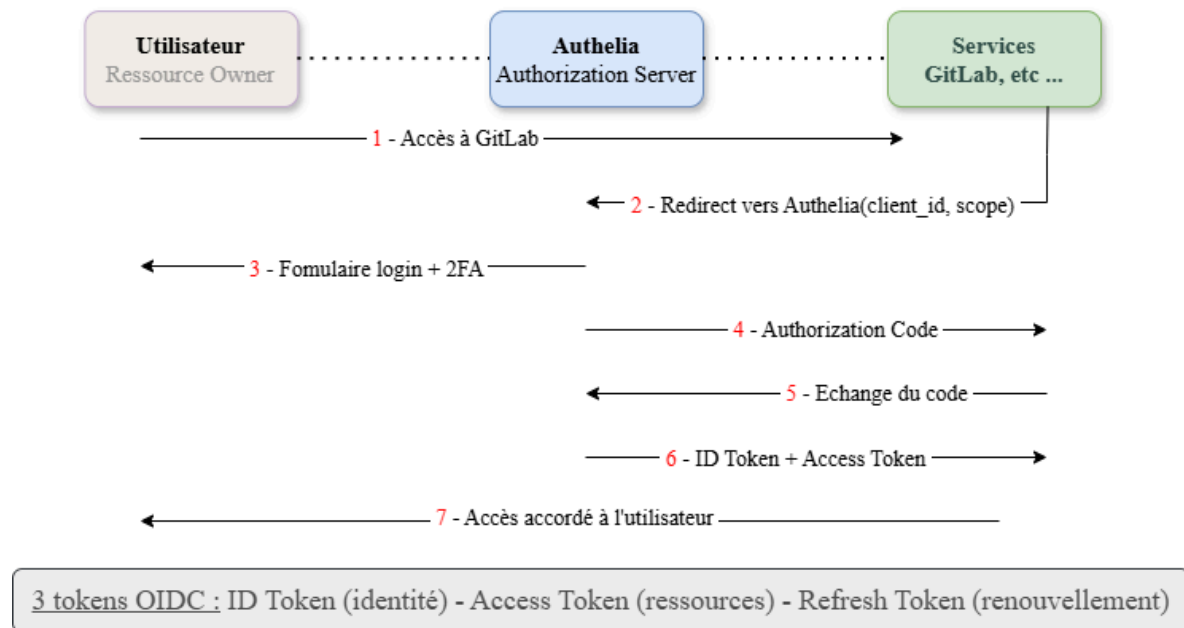


Figure 3 – Acteurs du protocole OIDC

Le flux OIDC qui s'applique ici est l'*Authorization Code Flow*, le plus sécurisé, utilisé pour les applications web.

2.2.2 – Authorization Code Flow

Le flux utilisé dans ce projet est l'*Authorization Code Flow*, considéré comme le plus sécurisé pour les applications web. Son déroulement est le suivant (voir Figure 3 ci-dessus)

- 1. L'utilisateur tente d'accéder à un service (ex : GitLab)
- 2. Le service redirige l'utilisateur vers Authelia avec un *client_id*, un *scope* et *redirect_uri*
- 3. L'utilisateur s'authentifie sur Authelia (login + 2FA)
- 4. Authelia émet un « Authorization Code » et redirige vers le service
- 5. Le service échange ce code contre un « ID Token » et un « Access Token »
- 6. L'accès est accordé à l'utilisateur

2.2.3 – Les tokens JWT (JSON Web Token)

OIDC utilise trois types de « Token » au format JWT pour transmettre les informations (voir *Figure 3 ci-dessus*) :

- ID Token – contient l'identité de l'utilisateur (nom, courriel, groupe, etc.)
- Access Token – autorise l'accès aux ressources protégées
- Refresh Token – permet le renouvellement des « Tokens » sans réauthentification

Token	Rôle	Utilisation
ID Token	Authentifier	Savoir qui est l'utilisateur
Access Token	Autoriser	Accéder aux services
Refresh Token	Renouveler	Eviter la reconnexion

2.2.4 – Implémentation d'OIDC

Une grande partie des solutions disponibles sur internet supporte l'authentification par OIDC. Cependant, toutes ne l'implémentent pas de la même manière.

Là où GitLab, Rocket.Chat et LibreDesk supportent nativement l'implémentation d'un SSO avec OIDC, Jenkins et Nextcloud demandent une phase de configuration en plus.

Pour Jenkins, nous devons installer des plug-ins pour que le service accepte et fasse le lien entre les connexions OIDC et les utilisateurs internes.

Pour le cas de Nextcloud, nous devons aller dans le magasin d'application de la solution pour y télécharger et activer celle nécessaire pour supporter OIDC.

2.3 – Authelia – Le cœur du système

Authelia est un serveur d'authentification et d'autorisation permettant de mettre en place un système SSO sécurisé avec double authentification. Il joue deux rôles simultanément, celui de portail d'authentification (login + 2FA), et celui de fournisseur OIDC pour les services connectés. Il s'intercale donc entre le reverse proxy et les services.

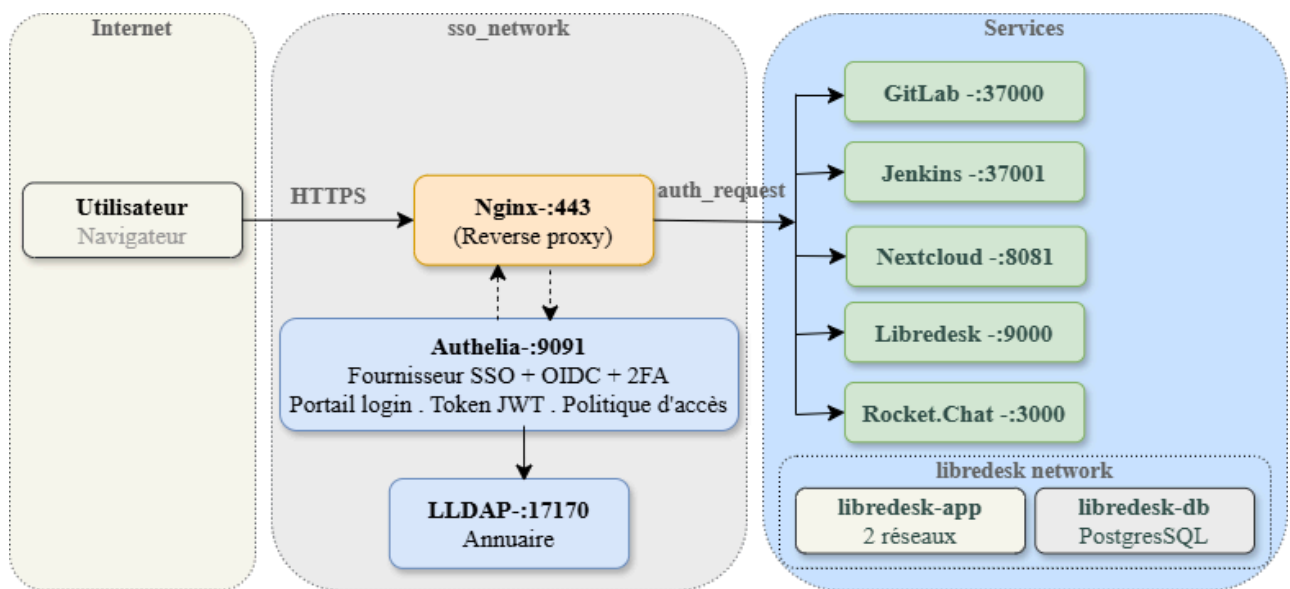


Figure 4 – Architecture complète SSO (Authelia)

L'architecture repose sur une approche conteneurisée avec Docker, permettant de déployer l'ensemble des services sans recourir à des machines distinctes. Tous les composants sont exécutés sous forme de conteneurs interconnectés, cela facilite la portabilité et la maintenance.

Le système est accessible via le domaine : **tp-sso.local**

Ce domaine centralise l'accès aux différents services à travers un reverse proxy (**Nginx**) sécurisé. Le réseau principal docker **sso_network** assure la communication globale, il contient le reverse proxy, **Athelia**, **LDAP**.

Libredesk_network est dédié uniquement aux composants de Libredesk.

Composants principaux

Composant	Rôle	Description
Nginx Proxy	Point d'entrée unique et routage des services	Reverse Proxy
Authelia	SSO + 2FA	Authentification
LLDAP	Gestion des utilisateurs	Annuaire
Services	Services applicatifs	Services consommés

En clair : ce qui se passe quand un utilisateur veut accéder à un service (exemple GitLab)

1. Il va sur : <https://gitlab.tp-sso.local>
2. **Nginx** intercepte la requête
3. Nginx demande à **Authelia** :
→ "Cet utilisateur est-il authentifié ?"

- 4. Si OUI → accès au service demandé
Si NON → login + 2FA, puis accès au service si succès

2.4 – Structure des dossiers

Pour l'organisation des dossiers, nous avons choisi une organisation avec les fichiers utiles à tout le projet à la racine puis un dossier ``infrastructure`` qui contient tous les volumes montés par les conteneurs Docker comme le montre [l'annexe A](#). Cela permet d'avoir les fichiers importants à la racine du projet puis d'avoir les dossiers relatifs à chaque conteneur à un endroit centralisé pour ne pas avoir à chercher à plusieurs endroits pour la configuration d'un service.

2.5 – Nginx comme reverse proxy

Dans cette architecture, Nginx remplit la fonction de point d'entrée unique pour l'ensemble du trafic réseau. Son rôle principal est d'assurer la liaison entre les clients externes et les services conteneurisés en gérant en premier lieu la terminaison TLS. Cette centralisation du déchiffrement HTTPS permet de garantir que les données sensibles ne circulent en clair que dans le réseau isolé `sso_network`, ce qui décharge les services applicatifs comme GitLab ou Jenkins de la gestion individuelle des certificats de sécurité.

En parallèle, Nginx assure un routage précis grâce au mécanisme de Virtual Hosting. En analysant les en-têtes d'hôte des requêtes entrantes, il redirige les flux vers les conteneurs appropriés en utilisant leurs noms de domaine pleinement qualifiés. Cela permet de séparer logiquement les services tout en partageant une seule adresse IP publique. Le reverse proxy agit ainsi comme un intermédiaire de confiance qui reçoit les demandes, valide la sécurité de la couche de transport et transmet les paquets aux applications de l'entreprise TechSolutions de manière transparente pour l'utilisateur final.

2.6 – Sécurité des systèmes SSO

La sécurité d'un système SSO repose sur la protection rigoureuse de son point d'accès central, car la compromission de ce dernier expose l'intégralité des ressources de l'entreprise. L'usage du protocole HTTPS constitue la première ligne de défense indispensable contre les attaques de type Man in the Middle. Dans ce scénario, un pirate intercepte la communication entre l'utilisateur et le serveur pour dérober les jetons d'authentification ou les cookies de session. Sans le chiffrement SSL/TLS, ces informations transitent en clair, ce qui permettrait à un individu malveillant de détourner une session active et d'accéder aux services comme GitLab ou Nextcloud sans même posséder les identifiants de la victime.

Pour pallier les risques liés au vol de mots de passe, l'authentification à deux facteurs joue un rôle déterminant. En imposant la saisie d'un code unique TOTP généré sur un appareil physique, le système neutralise les effets du hameçonnage. Même si un attaquant parvient à obtenir un mot de passe valide, il se retrouve dans l'incapacité de finaliser la connexion sans le second facteur détenu

par l'employé. Cette exigence transforme un mot de passe compromis en une donnée insuffisante, garantissant ainsi l'intégrité des accès aux infrastructures de TechSolutions.

Enfin, l'architecture doit prévenir les attaques par force brute, où des scripts automatisés testent des milliers de combinaisons d'identifiants. Authelia traite cette menace en appliquant un bannissement temporaire de l'adresse IP source après plusieurs échecs consécutifs. En complément, les risques de rejeu sont limités par l'attribution d'une durée de vie très courte aux jetons de session. Ces mesures techniques garantissent que seul un utilisateur légitime, muni de ses deux preuves d'identité, peut franchir le portail sécurisé pour accéder aux applications de l'organisation.

III – ORGANISATION DU GROUPE

3.1 – Répartition des rôles et attribution des tâches

Pour la répartition des tâches nous avons dressé une liste des différents points à traiter au cours du projet pour ensuite choisir celles qui nous intéressent le plus.

La répartition finale est la suivante :

- Implémentation de Nginx Proxy Manager, gestion DNS et SSL : Matthieu
- Implémentation d'Authelia et gestion des secrets : Louis
- Implémentation de GitLab et Jenkins : Alexandre
- Implémentation de Nextcloud, Rocket.Chat et Libredesk : Robin
- Préparation du rapport : Mohamed

Plusieurs points ont été réalisés en groupe selon les besoins notamment lors de l'intégration de fonctionnalités dans le projet final pour s'assurer que tout fonctionne correctement.

3.2 – Outils de collaboration

Nous avons utilisé Discord pour communiquer et s'échanger les fichiers critiques du projet tel que le fichier .env ou les clés et certificats utilisés.

Concernant le code du projet, tout a été mis en commun via un dépôt GitHub en faisant chacun une branche pour avancer sur notre périmètre avant de fusionner les différentes versions sur la branche principale.

IV – ARCHITECTURE DE L'INFRASTRUCTURE

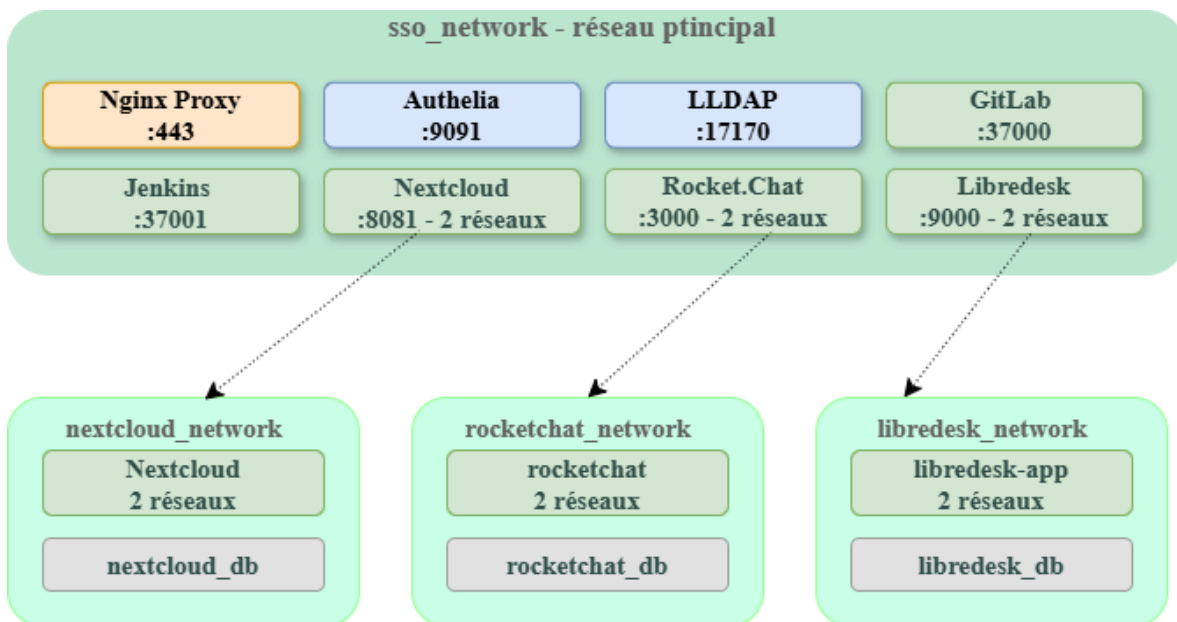
4.1 – Segmentation réseau Docker

Notre projet Docker comporte plusieurs réseaux pour garantir une segmentation sécuritaire des services et éviter des accès non autorisés.

Le réseau principal est nommé `sso_network`. Tous les conteneurs principaux des services ainsi que ceux de LDAP, Nginx proxy Manager et Authelia y sont connectés. En n'y connectant aucun autre conteneur, nous évitons d'avoir plusieurs bases de données en concurrence sur le même réseau et nous gardons notre architecture propre.

Les services Nextcloud et Rocket.Chat nécessitent tous les deux une base de données externalisée pour fonctionner. Nous avons donc créé deux autres réseaux `nextcloud_network` et `rocketchat_network` auxquels sont connectés respectivement les conteneurs `nextcloud/nextcloud_db` et `rocketchat/rocketchat_db`.

De plus, nous avons également isolé LDAP dans un réseau `ldap_network` pour ne l'exposer qu'à Authelia.



Le cas de LibreDesk est plus particulier car le service en lui-même est dans un docker-compose. Nous avons donc ajouté les conteneurs à notre fichier docker-compose dans réseau isolé `libredesk_network` avant d'exposer uniquement le conteneur racine sur le réseau `sso_network` pour le rendre accessible via Authelia.

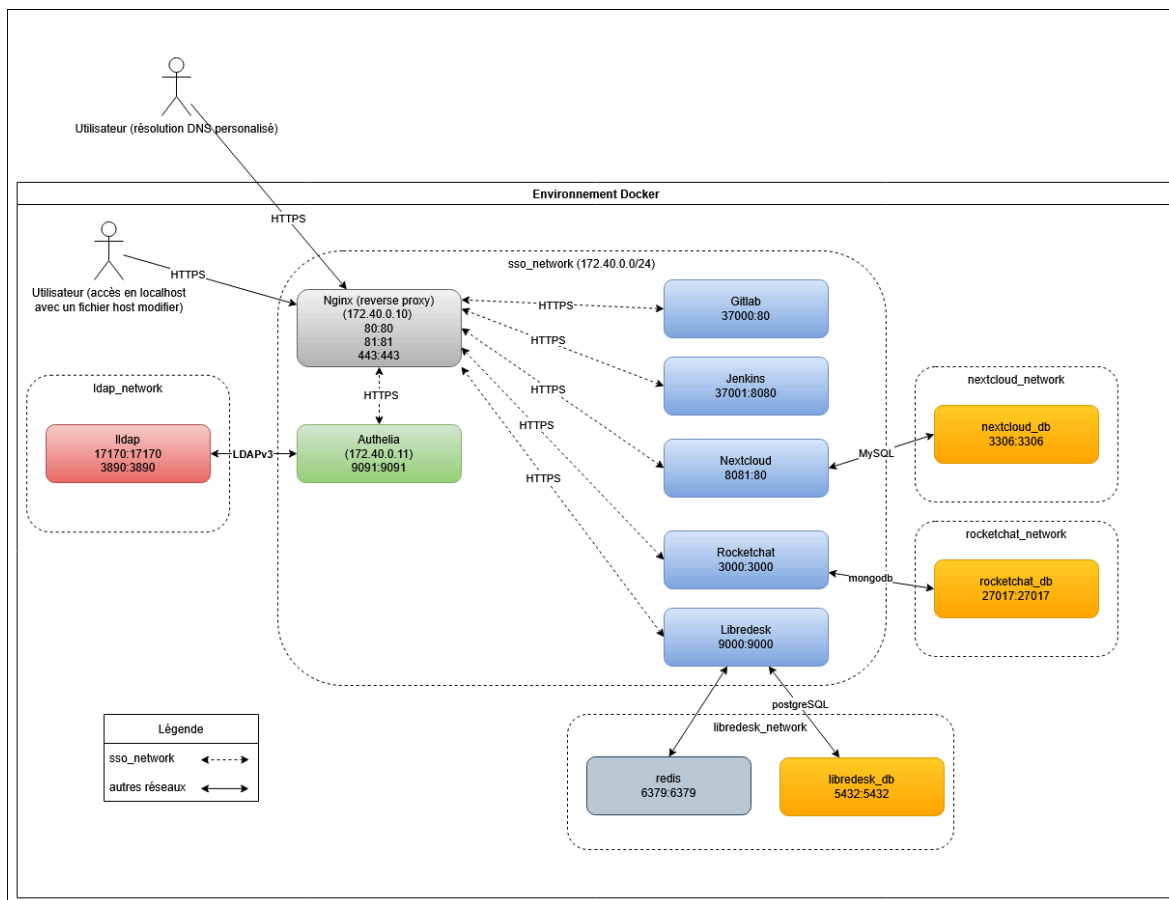
4.2 – FQDN et plan d'adressage

Pour ce projet, nous avons utilisé le nom de domaine `tp-sso.local` pour l'accès à tous nos services.

Les URLs racines sont formées avec le nom du service en minuscule et sans caractère spéciaux suivi du nom de domaine.

Service	FQDN	PORT	Protocole OIDC
Authelia	auth.tp-sso.local	9091	
GitLab	gitlab.tp-sso.local	37000	OIDC natif
Jenkins	jenkiens.tp-sso.local	37001	Plugin OIDC
Nextcloud	nextcloud.tp-sso.local	8080	App OIDC
Libredesk	libredes.tp-sso.local	9000	Plugin OIDC
Rocket.Chat	rocketchat.tp-sso.local	3000	OIDC natif

4.3 – Diagramme de flux réseau et ports



Les utilisateurs doivent accepter les certificats qui sont auto-signés à travers leurs navigateurs. Pour le fichier host a modifié, il doit contenir les lignes suivantes :

127.0.0.1 auth.tp-sso.local

127.0.0.1 nginx.tp-sso.local

127.0.0.1 gitlab.tp-sso.local

127.0.0.1 nextcloud.tp-sso.local

127.0.0.1 rocketchat.tp-sso.local

127.0.0.1 libredesk.tp-sso.local

127.0.0.1 jenkins.tp-sso.local

4.4 – Choix technique et justification

Nous avons choisi de monter notre infrastructure avec docker pour des raisons d'économie de ressources, de simplicité et de portabilité.

Comme ce projet est composé de nombreux services, nous avons estimé que, même avec les configurations minimales, tout monter sur des machines virtuelles ne serait pas raisonnable.

Utiliser des conteneurs est moins demandant en ressources et plus facilement modulable et ajustable en fonction de nos besoins.

De plus, le fait que la colonne vertébrale de notre projet soit un fichier docker compose nous assure de tous utiliser la même version d'image pour chaque service, d'avoir les mêmes chemins pour accéder aux fichiers dans le projet ou encore les mêmes ajustements à faire lors de corrections sur des fichiers non pris en compte sur le dépôt Git.

V – GESTION DES IDENTITÉS ET DES ACCÈS

5.1 – Nomenclature CIP des utilisateurs

Chaque employé de l'entreprise se voit attribué un code à 4 caractères qui suit le format suivant :

[Initiale du prénom][Initiale du nom][2 chiffres]

Exemples : Alice Dupont → ad01; Bob Martin → bm01; François Petit → fp01

Ces CIP serviront de nom d'utilisateur LDAP et de login SSO pour tous les services.

5.2 – Annuaire OpenLDAP, Groupes et permissions associées

Pour donner les autorisations d'accès à un service à un employé, nous avons créé des groupes pour chaque service dans LDAP sous le format ``nextcloud_user`` puis ajouté les utilisateurs devant accéder au service en question dans le groupe.

Groupe LDAP	Membres	Accès autorisés
gitlab_users	ad01, bm01, cl01, dm01, eb01, fp01, hl01	gitlab.tp-sso.local
jenkins_users	ad01, bm01, dm01, fp01, hl01	jenkins.tp-sso.local
libredesk_users	tous les utilisateurs	libredesk.tp-sso.local
nextcloud_users	ad01, bm01, dm01, fp01, gr01, if01, jg01	nextcloud.tp-sso.local
rocketchat_users	tous les utilisateurs	rocketchat.tp-sso.local

5.3 – Politique d'accès Authelia

La politique d'accès a été configurée pour respecter une approche de "**moindre privilège**". Par défaut, l'accès à toute ressource non explicitement définie est rejeté (*default_policy: deny*).

→ Accès par groupe : Les accès sont restreints selon les groupes définis dans LLDAP.

Par exemple, seuls les utilisateurs appartenant au groupe *gitlab_user* peuvent accéder au domaine *gitlab.tp-sso.local*.

- Niveaux d'authentification : Pour les services critiques comme GitLab ou Jenkins, une politique de deux facteurs est imposée, forçant l'utilisation du TOTP après le mot de passe.

Le portail authelia lui-même est configuré en *bypass* pour éviter les boucles infinies de redirection lors de l'accès à la mire de login.

Le service Rocketchat a un paramètre de vérification à deux facteurs par email qui est configuré de base. Il faut le désactiver car Authelia se charge déjà de cette partie et cela pourrait bloquer l'utilisateur.

VI – IMPLÉMENTATION

6.1 – Mise en place de l'environnement Docker

Notre environnement Docker tient dans le fichier *docker-compose.yml*. Nous y trouvons l'organisation des réseaux, les différents conteneurs du projet avec leurs configuration, leurs variables ainsi que les ports qu'ils exposent sur le localhost.

En plus du fichier *docker-compose.yml*, il y a à la racine du projet le fichier *.env* ainsi que le dossier *infrastructure*. Ils contiennent respectivement toutes les variables d'environnement utilisées par les conteneurs et les dossiers pour les volumes montés par chaque conteneurs du projet.

6.2 – Configuration de Nginx

L'implémentation de la couche réseau repose sur le déploiement d'un conteneur Nginx agissant comme unique passerelle entre le réseau externe et l'infrastructure interne. Pour répondre aux exigences de clarté et de segmentation du cahier des charges, nous avons défini des FQDN pour chaque service selon une nomenclature standardisée. Chaque application est ainsi accessible via une adresse spécifique de type `[service].tp-sso.local`. Cette organisation permet à Nginx de diriger le trafic vers les bons conteneurs au sein du réseau isolé `sso_network`.

Conformément aux exigences de sécurité, nous avons forcé l'utilisation de SSL et donc de HTTPS sur le reverse proxy pour l'intégralité de ces domaines. Pour ce faire, nous avons généré des certificats TLS auto-signés couvrant chaque service. Cette approche garantit le chiffrement des données sensibles, comme les identifiants et les jetons de session, durant leur transit. Toutefois, l'utilisation de certificats auto-signés engendre systématiquement une alerte de sécurité dans le navigateur web, celui-ci ne pouvant vérifier l'identité du serveur auprès d'une autorité de certification reconnue. Dans le cadre de ce projet, nous avons dû valider manuellement ces exceptions dans nos navigateurs pour permettre l'accès aux interfaces tout en maintenant un tunnel de communication chiffré.

6.3 – Déploiement d'Authelia

Le déploiement a nécessité des ajustements spécifiques pour fonctionner dans un environnement de développement local sans HTTPS natif immédiat :

- **Configuration des Cookies** : Pour permettre le test sur le port `9091`, nous avons dû forcer l'option `insecure: true` au niveau des cookies de session afin que le navigateur accepte les jetons via HTTP simple.
- **Clés Cryptographiques (JWKS)** : nous avons généré et intégré une paire de clés RSA au format PEM pour la signature des jetons OIDC. Une attention particulière a été portée au formatage (indentation YAML et respect des 5 tirets du bloc PEM) pour éviter les erreurs de parsing au démarrage.

6.4 – Intégration OIDC par service

6.4.1 – GitLab

Pour l'implémentation de Gitlab, on a utilisé l'image du community edition pour bénéficier des fonctionnalités basiques. Par la suite, on initialise une première fois le conteneur et des fichiers de configurations devraient apparaître. Parmi ces fichiers, on a gitlab.rb. Celui-ci va nous permettre d'activer l'authentification via Authelia grâce au module omniauth. De plus, ce fichier est monté dans un volume, donc en effectuant une initialisation du conteneur, la configuration sera en place. Lorsque l'on accède à Gitlab, un bouton sera présent avec le label "Authelia", en cliquant dessus, on se voit rediriger vers Authelia pour s'authentifier.

6.4.2 – Jenkins

Pour Jenkins, on va tout d'abord mettre en place une image personnalisée qui contiendra les plugins oic-auth, configuration-as-code et role-strategy. Le module configuration-as-code permettra d'initialiser un fichier de configuration (jenkins.yaml) qui aura été monté au préalable comme volume et cette configuration contiendra les éléments qui permet à Jenkins de se configurer automatiquement sans avoir besoin passer par le GUI.

Dans le fichier de configuration, on peut retrouver les paramètres de l'ID client, le secret client et le lien du fournisseur. On retrouve aussi d'autres paramètres permettant la redirection automatique. Une fois les configurations mises en place, l'accès à Jenkins va directement renvoyer vers Authelia pour authentification.

6.4.3 – Nextcloud

Le service Nextcloud ne fonctionne pas avec le conteneur applicatif seul. Il faut donc créer un second conteneur qui héberge une base de données MariaDB qui sera associée au conteneur applicatif. Ces deux conteneurs communiquent via le réseau `nextcloud_network` et seront les seuls à communiquer sur ce réseau.

Lors du premier démarrage de Nextcloud, il faut se connecter à l'interface web du service pour créer un compte administrateur et commencer à utiliser le stockage. Dans une optique d'automatiser un maximum la configuration des services, nous avons découvert qu'il est possible de faire toute la configuration en passant par des commandes shell dans un terminal. Nous avons donc créé un script shell `setup.sh` ([Annexe B](#)) qui contient la commande `php occ maintenance:install` ainsi que tous les paramètres nécessaires comme le login et le mot de passe admin et BDD.

Concernant la configuration pour prendre en charge l'OIDC, Nextcloud ne l'a pas nativement. Nous devons donc installer une application à partir du magasin d'application Nextcloud. Cette opération se fait généralement via l'interface web mais il est également possible de le faire via la commande `php occ app:install <application>` pour l'installer et `php occ app:enable <application>` pour l'activer. Nous avons donc ajouté ces commandes pour l'application `oidc_login` dans le script

``setup.sh`` pour s'assurer que la prise en charge d'OIDC est bien activée au démarrage du conteneur.

Pour faire en sorte que le script de setup soit exécuté au lancement du conteneur, nous avons monté un volume sur le dossier ``/docker-entrypoint-hooks.d/before-starting`` et ajouter le script shell à celui-ci. Cela va faire en sorte que le script tourne juste après l'initialisation du conteneur, juste avant de rendre le service disponible pour utilisation.

Une fois Nextcloud configuré et prêt à l'emploi, nous devons ajouter une section à un fichier de configuration pour ajouter Authelia en tant que fournisseur d'identité pris en charge par le service.

6.4.4 – Rocket.Chat

Tout comme Nextcloud, Rocket.Chat a besoin d'une base de données pour fonctionner. Nous avons donc suivi le même principe en créant un conteneur MongoDB et en le connectant au conteneur applicatif via le réseau ``rocketchat_network``.

Une fois la base de données couplée, le conteneur démarre correctement et nous devons donc maintenant aller sur l'interface web pour créer un compte admin et activer l'environnement. Cette action n'est pas réalisable en ligne de commande car Rocket.Chat demande une adresse électronique réelle pour envoyer un courriel, valider l'adresse et activer l'environnement.

Concernant la prise en charge OIDC, Rocket.Chat le fait nativement mais une nouvelle fois, il n'y a pas de possibilité de le faire en ligne de commande. Nous devons donc aller sur l'interface web pour ajouter manuellement un fournisseur d'identité.

6.4.5 – LibreDesk

Le cas de LibreDesk est un peu différent des autres services. En effet, la solution est livrée dans un fichier docker-compose ce qui fait que nous avons du fusionner le fichier LibreDesk avec le fichier qui contient tout le reste de notre infrastructure. Une fois la fusion achevée, nous avons isolé les conteneurs LibreDesk sur le réseau ``libredesk_network`` et uniquement exposé le conteneur applicatif sur le réseau ``sso_network``.

Lors du premier démarrage de LibreDesk, nous devons aller sur l'interface web pour choisir le mot de passe du compte admin avant d'utiliser le service. Cependant, il est possible de définir ce mot de passe dans la section ``environment`` du conteneur applicatif. Cela permet de directement passer à la configuration OIDC du service.

LibreDesk supporte nativement OIDC, cependant, il n'est pas possible de faire cette configuration automatiquement. Nous devons donc aller sur l'interface admin pour ajouter Authelia comme fournisseur d'identité pour ce service.

6.4.6 – Authelia

Du côté d'Authelia, l'intégration d'OIDC est marquée par la création de clients dans le fichier de configuration. Chaque client est associé à un service et contient les informations dont le service en question aura besoin pour ajouter Authelia en tant que fournisseur d'identité de confiance (Voir [Annexe C](#)).

Un client contient un identifiant client, le hash du secret utilisé par le service, la politique de validation de connexion (MFA, refus...), les informations qui seront utilisées pour vérifier la connexion (openid, email, groupe, etc) ou encore l'adresse de redirection une fois l'authentification validée.

6.5 – Gestion des variables d'environnement et secrets

Les variables d'environnement sont toutes stockées dans le fichier `.env`. Cela permet qu'elles ne soient pas en clair dans les fichiers de configuration des services.

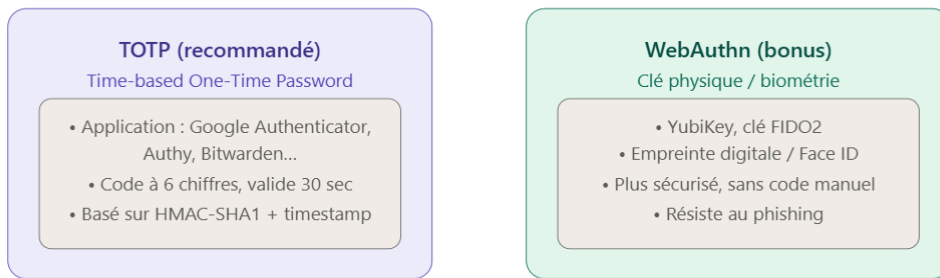
Les variables utilisées sont les suivantes :

- LLDAP_JWT_SECRET
- LLDAP_ADMIN_PASSWORD
- AUTHELIA_JWT_SECRET
- AUTHELIA_STORAGE_KEY
- AUTHELIA_SESSION_SECRET
- AUTHELIA_OIDC_PRIVATE_KEY
- AUTHELIA_GITLAB_SECRET
- AUTHELIA_JENKINS_SECRET
- AUTHELIA_NEXTCLOUD_SECRET
- AUTHELIA_ROCKETCHAT_SECRET
- AUTHELIA_JIRA_SECRET
- NEXTCLOUD_ADMIN_USER
- NEXTCLOUD_ADMIN_PASSWORD
- NEXTCLOUD_MYSQL_ROOT_PASSWORD
- NEXTCLOUD_MYSQL_USER_PASSWORD
- LIBREDESK_SYSTEM_USER_PASSWORD
- POSTGRES_DB
- POSTGRES_USER
- POSTGRES_PASSWORD

6.6 – Configuration du 2FA

Authelia supporte nativement les deux méthodes d'authentification suivantes :

Méthodes 2FA supportées par Authelia



Pour des raisons pratiques et matérielles, nous avons choisi d'opter pour la méthode TOTP. Cela permet de tester facilement et rapidement plusieurs utilisateurs. Il nous suffit d'associer notre téléphone avec une application type Google Authenticator lors de la première authentification pour avoir les codes éphémères.

VII – POINTS DÉPASSANT LE CAHIER DE CHARGES

7.1 – Gestion sécurisée des secrets par injection

Au-delà de la simple utilisation d'un fichier `.env`, nous avons mis en place un système de fichiers secrets montés (`secrets/*`).

Cela permet à Authelia de lire ses clés cryptographiques et ses mots de passe de base de données directement depuis le système de fichiers du conteneur, réduisant ainsi le risque de fuite via l'inspection des processus Docker (`docker inspect`).

La segmentation des réseaux pour les services et leurs base de données ont été aussi juger comme intéressant pour isoler les données présent dans les services et la possibilité d'une intrusion en passant par le réseau `sso_network`.

VIII – POINTS BLOQUANTS ET MITIGATIONS

8.1 – Problèmes rencontrés

→ Contrainte de licence

JIRA et **Confluence** sont des outils propriétaires nécessitant des licences payantes, cela rend moins adaptés à un projet académique (pas de budget).

→ Validation stricte du schéma HTTPS

Les versions récentes d'Authelia (v4.38+) refusent de démarrer si les URLs de redirection ou de portail n'utilisent pas le protocole sécurisé [https](#).

→ Blocs PEM malformâtes

Lors de l'intégration, des erreurs de formatage dans le bloc `jwks` (nombre de tirets incorrects ou espaces superflus) ont causé des échecs critiques au démarrage.

→ Certificats SSL pour HTTPS

Lors de la mise en commun, tous les services ne pouvaient pas accéder à Authelia. Le problème provenait de la validation du certificat qui a été auto-sign. Il a donc fallu rectifier des configurations pour certains services pour faire en sorte qu'il puisse valider ce certificat.

Pour Gitlab : On a monté le certificat dans le dossier du conteneur qui contient tous les certificats et le service va se charger de les lire automatiquement.

Pour Jenkins : On a utilisé le Dockerfile pour effectuer une copie du certificat pour ensuite les mettre dans le répertoire des certificats et exécuter une commande pour mettre à jour les certificats valides.

Pour Nexclouds : On a utilisé un Dockerfile pour faire une copie du certificat et l'ajouter dans le dossier des certificats et exécuter une commande pour mettre à jour les certificats.

Pour Rocket.chat : Une rectification dans le docker compose pour ajouter un volume qui va monter le certificat directement dans le dossier cible.

Pour LibreDesk : Celui-ci n'acceptait pas les certificats avec des Common Names mais plutôt des Subject Names Alternatives. Il a donc fallu générer des nouveaux certificats et les insérer dans le dossier des certificats de libredesk.

→ Résolution DNS des conteneurs

Un des problèmes rencontrés a été de configurer les fichiers hosts des conteneurs pour faire en sorte qu'il puisse trouver l'adresse du proxy (172.40.0.10) à travers son nom d'hôte. Cela permet par la suite au service de savoir où se trouve le proxy qui va se charger des redirections par la suite.

8.2 – Solutions apportées

→ Contrainte de licence

Une première solution visait à remplacer **JIRA** et **Confluence** par **Plane** mais l'implémentation d'un SSO personnalisé sur ce service s'est révélée être une fonctionnalité premium. Finalement, ils ont été remplacés par une solution open-source (**Libredesk**); Ce changement s'explique principalement par les contraintes de licences payantes. Libredesk permet de couvrir les besoins initialement prévus avec *JIRA* et *Confluence*.

→ Utilisation de flags d'insécurité

L'ajout de la variable `AUTHelia_SESSION_COOKIES_INSECURE=true` a permis de contourner temporairement la restriction *HTTPS* pour la phase de développement.

→ Normalisation YAML

Re-formatage strict des clés privées en respectant l'indentation YAML et les standards PEM (5 tirets au début et à la fin).

IX – CONCLUSION

9.1 – Notions apprises

Ce projet nous a permis de comprendre en profondeur comment fonctionne Docker ainsi que l'importance de configurer correctement les conteneurs d'un projet.

Il s'agit d'un bon projet pour permettre une bonne compréhension du fonctionnement d'un SSO et de comment mettre en place un environnement qui serait déployable grâce à des conteneurs Docker qui est un outil permettant une grande portabilité d'application et d'environnement comme le nôtre.

Nous avons pu comprendre comment fonctionne un SSO en allant en profondeur sur les documentations OIDC mais également nous rendre compte qu'il n'y a pas de solution unique pour ce type de mécanisme et que chaque service l'intègre à sa manière.

Nous nous sommes également rendu compte de l'importance de suivre l'évolution de la disponibilité des services proposés par les sociétés dont nous sommes clients pour éviter de se retrouver dans une impasse concernant les solutions intégrées à notre projet.

9.2 – Perspectives d'amélioration

Pour améliorer notre projet, nous pouvons mettre en place une solution qui automatisera la rotation des tokens de connexion.

Nous pouvons également chercher à plus automatiser l'initialisation des conteneurs pour avoir moins d'interventions à faire lors du premier lancement du docker-compose.

Un autre aspect d'amélioration aurait pu être l'intégration avec un LDAP plus répandu en entreprise comme l'Active Directory de Windows server. Cela aurait pu nous permettre d'expérimenter sur des infrastructures plus répandues.

Il aurait pu être aussi possible d'améliorer la configuration pour faire en sorte de uniquement utiliser le fichier docker compose pour le déploiement. Cela aurait pu grandement améliorer la portabilité de l'environnement même si nos fichier reste quand même une infrastructure portable.

X – RÉFÉRENCES

<https://www.authelia.com/integration/openid-connect/clients/rocket-chat/>

<https://www.authelia.com/integration/openid-connect/clients/nextcloud/>

https://docs.nextcloud.com/server/stable/admin_manual/installation/command_line_installation.html

<https://docs.libredesk.io/getting-started/installation>

<https://docs.gitlab.com/integration/saml/>

<https://learn.microsoft.com/en-us/powershell/>

<https://docs.docker.com/reference/cli/docker/compose/>

<https://docs.docker.com/compose/how-tos/networking/>

XI – Annexes

Annexe A — Arborescence du projet

.env	← Fichier contenant les variables critiques du projet
docker-compose.yml	← Fichier contenant la configuration de tous les conteneurs
cert.pem	← Certificat nécessaire pour le protocole SSL
key.pem	← Clé nécessaire pour le protocole SSL
README.md	← Instructions pour les étapes non faisables en ligne de commande
└─ infrastructure	← Dossier racine pour tous les volumes montés par les conteneurs
└─ authelia	
├─ configuration.yml	← Configuration complète d'Authelia
├─ db.sqlite3	
└─ secrets	← Dossier contenant les secrets des clients OIDC
└─ Gitlab	
├─ config	
├─ gitlab.rb	← Fichier à modifier pour intégrer le support d'OIDC
└─ data	
└─ Jenkins	
├─ config.yml	← Fichier de config de base pour Jenkins
├─ Dockerfile	← Fichier pour automatiser l'installation des plug-ins au démarrage
├─ jenkins.yaml	← Fichier à créer pour intégrer le support d'OIDC
└─ data	
└─ libredesk	
├─ config.toml	← Fichier de config nécessaire pour le lancement de libredesk
└─ uploads	
└─ lldap	
├─ lldap_config.toml	← Fichier de config de base pour LDAP
└─ users.db	← Base de donnée contenant les utilisateurs et les groupes créés
└─ nextcloud	
├─ before-starting	← Dossier pour les hooks Docker de type `before-starting`
├─ setup.sh	← Script pour initialiser correctement Nextcloud
├─ data	
├─ config	
├─ config.php	← Fichier à modifier pour intégrer le support d'OIDC
└─ db	
└─ nginx	
├─ data	
├─ custom_ssl	
├─ npm-4	
├─ fullchain.pem	← Certificat pour les communications TLS
├─ privkey.pem	← Clé pour les communications TLS
└─ nginx	
└─ proxy_host	← Dossier contenant les configurations des proxys créés
└─ rocketchat	
├─ db	
└─ uploads	

Annexe B — Script d'initialisation Nextcloud `setup.sh`

```
#!/bin/bash
# Installation de Nextcloud
echo "Installation de Nextcloud..."
php occ maintenance:install \
  --database "mysql" \
  --database-host "$MYSQL_HOST" --database-name "$MYSQL_DATABASE" \
  --database-user "$MYSQL_USER" --database-pass "$MYSQL_PASSWORD" \
  --admin-user "$NEXTCLOUD_ADMIN_USER" \
  --admin-pass "$NEXTCLOUD_ADMIN_PASSWORD"
# Installation de l'application OIDC
echo "Installation de l'application OIDC..."
php occ app:install oidc_login
php occ app:enable oidc_login
```

Annexe C — Client type dans la configuration Authelia

```
# NEXTCLOUD
- client_id: 'nextcloud'
  client_name: 'NextCloud'
  client_secret: '$pbkdf2-sha512$310000$mSlmsfp1AJjQyvihqjHREA$Js...'
  public: false
  authorization_policy: 'two_factor'
  require_pkce: true
  pkce_challenge_method: 'S256'
  claims_policy: 'nextcloud_userinfo'
  redirect_uris:
    - 'https://nextcloud.tp-sso.local/apps/oidc_login/oidc'
  scopes:
    - 'openid'
    - 'profile'
    - 'email'
    - 'groups'
    - 'nextcloud_userinfo'
  response_types:
    - 'code'
  grant_types:
    - 'authorization_code'
  access_token_signed_response_alg: 'none'
  userinfo_signed_response_alg: 'none'
  token_endpoint_auth_method: 'client_secret_basic'
```